

Finding a Close Point in a Lattice

Keng-Yu Chen

August 7, 2026

Note: This document was auto-generated by AI from a Markdown note on Notion. Its correctness has *not* yet been verified — please check the reference papers and all definitions, algorithms, and proofs before relying on them.

1 Problem Setting

The lattice $\mathcal{L}(\mathbf{B})$ of some matrix $\mathbf{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$, where the $\mathbf{b}_i$'s are column vectors, is

$$\mathcal{L}(\mathbf{B}) = \left\{ \mathbf{B}\mathbf{x} = \sum_{i=1}^n x_i \mathbf{b}_i : x_i \in \mathbb{Z} \right\}.$$

The goal is that, given some vector $\mathbf{c}$, find a vector $\mathbf{z} \in \mathcal{L}(\mathbf{B})$ which is close to $\mathbf{c}$, with closeness measured by Euclidean distance.

2 Preliminaries

Definition 1 (Gram–Schmidt Orthogonalization). The Gram–Schmidt orthogonalization (GSO) of a full-rank matrix $\mathbf{B}$ is $\tilde{\mathbf{B}} = \{\tilde{\mathbf{b}}_1, \dots, \tilde{\mathbf{b}}_n\}$ where

$$\tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j=1}^{i-1} \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle} \tilde{\mathbf{b}}_j.$$

We can write $\mathbf{B} = \tilde{\mathbf{B}}\mathbf{U}$, or equivalently $\mathbf{B}^t = \mathbf{L}\tilde{\mathbf{B}}^t$, for some upper-triangular matrix $\mathbf{U}$ or lower-triangular matrix $\mathbf{L}$ with 1's on their diagonals, where

$$\mathbf{U} = \begin{bmatrix} 1 & \frac{\langle \mathbf{b}_2, \tilde{\mathbf{b}}_1 \rangle}{\langle \tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_1 \rangle} & \frac{\langle \mathbf{b}_3, \tilde{\mathbf{b}}_1 \rangle}{\langle \tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_1 \rangle} & \dots & \frac{\langle \mathbf{b}_n, \tilde{\mathbf{b}}_1 \rangle}{\langle \tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_1 \rangle} \\ 0 & 1 & \frac{\langle \mathbf{b}_3, \tilde{\mathbf{b}}_2 \rangle}{\langle \tilde{\mathbf{b}}_2, \tilde{\mathbf{b}}_2 \rangle} & \dots & \frac{\langle \mathbf{b}_n, \tilde{\mathbf{b}}_2 \rangle}{\langle \tilde{\mathbf{b}}_2, \tilde{\mathbf{b}}_2 \rangle} \\ 0 & 0 & 1 & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 1 \end{bmatrix}.$$

Definition 2 (Gaussian Function). The Gaussian function in n dimensions is

$$\rho_{s,\mathbf{c}}(\mathbf{x}) = \exp\left(-\pi \frac{\|\mathbf{x} - \mathbf{c}\|^2}{s^2}\right).$$

We may omit $\mathbf{c}$ when $\mathbf{c} = \mathbf{0}$.

Definition 3 (Discrete Gaussian Distribution). The discrete Gaussian distribution over a lattice Λ is

$$D_{\Lambda,s,\mathbf{c}}(\mathbf{x}) = \frac{\exp\left(-\pi \frac{\|\mathbf{x} - \mathbf{c}\|^2}{s^2}\right)}{\sum_{\mathbf{v} \in \Lambda} \exp\left(-\pi \frac{\|\mathbf{v} - \mathbf{c}\|^2}{s^2}\right)} = \frac{\rho_{s,\mathbf{c}}(\mathbf{x})}{\rho_{s,\mathbf{c}}(\Lambda)}.$$

3 Babai's Round-off Algorithm [Bab86]

Algorithm 1 Babai's Round-off Algorithm

Input: Basis $\mathbf{B}$, target vector $\mathbf{c}$

Output: Vector $\mathbf{z} \in \mathcal{L}(\mathbf{B})$ close to $\mathbf{c}$

- 1: Compute $\mathbf{B}^{-1}$
 - 2: $\mathbf{v} \leftarrow \mathbf{B} \cdot \lfloor \mathbf{B}^{-1} \mathbf{c} \rfloor$
 - 3: **return** $\mathbf{v}$
-

Note that

$$\mathbf{c} - \mathbf{v} = \mathbf{B} \cdot \mathbf{B}^{-1} \mathbf{c} - \mathbf{B} \cdot \lfloor \mathbf{B}^{-1} \mathbf{c} \rfloor \in \mathbf{B} \cdot [-1, 1]^n.$$

4 Nearest-Plane Algorithm [Kle00]

Algorithm 2 Nearest-Plane Algorithm

Input: Basis $\mathbf{B}$, target vector $\mathbf{c}$

Output: Vector $\mathbf{v}_0 \in \mathcal{L}(\mathbf{B})$ close to $\mathbf{c}$

- 1: $\mathbf{c}_n \leftarrow \mathbf{c}$, $\mathbf{v}_n \leftarrow \mathbf{0}$
 - 2: **for** $i = n, \dots, 1$ **do**
 - 3: $c'_i \leftarrow \frac{\langle \mathbf{c}_i, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle}$
 - 4: $z_i \leftarrow \lfloor c'_i \rfloor$
 - 5: $\mathbf{c}_{i-1} \leftarrow \mathbf{c}_i - z_i \mathbf{b}_i$, $\mathbf{v}_{i-1} \leftarrow \mathbf{v}_i + z_i \mathbf{b}_i$
 - 6: **end for**
 - 7: **return** $\mathbf{v}_0$
-

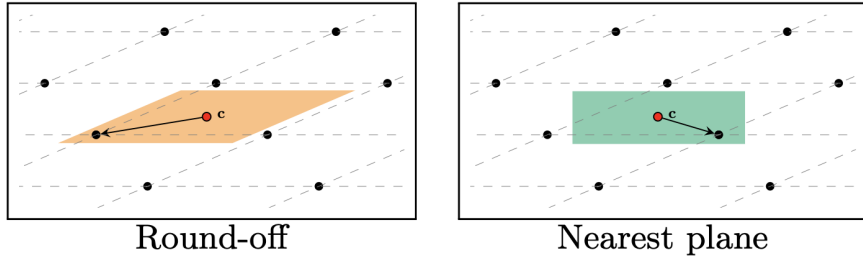


Figure 1: Illustration of the nearest-plane algorithm, from [DP16].

Theorem 1.

$$\mathbf{v}_0 - \mathbf{c} = \sum_{i=1}^n (z_i - c'_i) \tilde{\mathbf{b}}_i \in \tilde{\mathbf{B}} \cdot \left[-\frac{1}{2}, \frac{1}{2} \right]^n.$$

Proof. We claim that for $j = 1, 2, \dots, n$,

$$\mathbf{v}_0 - \mathbf{v}_j - \pi_j(\mathbf{c}_j) = \sum_{i=1}^j (z_i - c'_i) \tilde{\mathbf{b}}_i,$$

where $\pi_j(\cdot)$ is the orthogonal projection onto $\text{span}(\mathbf{b}_1, \dots, \mathbf{b}_j)$,

$$\pi_j(\mathbf{t}) = \sum_{k=1}^j \frac{\langle \mathbf{t}, \tilde{\mathbf{b}}_k \rangle}{\langle \tilde{\mathbf{b}}_k, \tilde{\mathbf{b}}_k \rangle} \tilde{\mathbf{b}}_k, \quad \tilde{\mathbf{b}}_j = \mathbf{b}_j - \pi_{j-1}(\mathbf{b}_j).$$

Observe that

$$\pi_j(\mathbf{c}_j) = c'_j \tilde{\mathbf{b}}_j + \pi_{j-1}(\mathbf{c}_j), \quad \tilde{\mathbf{b}}_j = \mathbf{b}_j - \pi_{j-1}(\mathbf{b}_j).$$

Note that when $j = n$, the above equation becomes what we desire, since $\mathbf{v}_n = \mathbf{0}$ and $\pi_n(\mathbf{c}_n) = \pi_n(\mathbf{c}) = \mathbf{c}$. We prove this by induction. For $j = 1$,

$$\mathbf{v}_0 - \mathbf{v}_1 - \pi_1(\mathbf{c}_1) = z_1 \mathbf{b}_1 - \pi_1(\mathbf{c}_1) = z_1 \tilde{\mathbf{b}}_1 - c'_1 \tilde{\mathbf{b}}_1.$$

Now suppose the equation holds for all $j < k$. For $j = k$,

$$\begin{aligned} \mathbf{v}_0 - \mathbf{v}_j - \pi_j(\mathbf{c}_j) &= \mathbf{v}_0 - (\mathbf{v}_{k-1} - z_k \mathbf{b}_k) - \pi_{k-1}(\mathbf{c}_k) + c'_k \tilde{\mathbf{b}}_k \\ &= \mathbf{v}_0 - \mathbf{v}_{k-1} + z_k \mathbf{b}_k - \pi_{k-1}(\mathbf{c}_{k-1} + z_k \mathbf{b}_k) - c'_k \tilde{\mathbf{b}}_k \\ &= \mathbf{v}_0 - \mathbf{v}_{k-1} - \pi_{k-1}(\mathbf{c}_{k-1}) + z_k (\mathbf{b}_k - \pi_{k-1}(\mathbf{b}_k)) - c'_k \tilde{\mathbf{b}}_k \\ &= \sum_{i=1}^{k-1} (z_i - c'_i) \tilde{\mathbf{b}}_i + z_k \tilde{\mathbf{b}}_k - c'_k \tilde{\mathbf{b}}_k. \end{aligned}$$

□

4.1 Another Version via $\mathbf{U}$

Algorithm 3 Nearest-Plane Algorithm (via $\mathbf{U}$)

Input: Basis $\mathbf{B}$, target vector $\mathbf{c}$

Output: Vector $\mathbf{v} \in \mathcal{L}(\mathbf{B})$ close to $\mathbf{c}$

- 1: Find $\mathbf{t} = \mathbf{B}^{-1}\mathbf{c}$ such that $\mathbf{B}\mathbf{t} = \mathbf{c}$
 - 2: **for** $i = n, \dots, 1$ **do**
 - 3: $t'_i \leftarrow t_i + \sum_{j>i} \mathbf{U}_{ij}(t_j - z_j)$
 - 4: $z_i \leftarrow \lfloor t'_i \rfloor$
 - 5: **end for**
 - 6: **return** $\mathbf{v} = \sum_{i=1}^n z_i \mathbf{b}_i$
-

Proof of Equivalence. Algorithms 2 and 3 compute the same thing. We show that $t'_i = c'_i$. Note that

$$c'_i = \frac{\langle \mathbf{c} - \sum_{j>i} z_j \mathbf{b}_j, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle} = \frac{\langle \mathbf{c}, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle} - \frac{\langle \sum_{j>i} z_j \mathbf{b}_j, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle}.$$

Also, $\mathbf{U}_{ij} = \frac{\langle \mathbf{b}_j, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle}$, so

$$\sum_{j>i} \mathbf{U}_{ij} z_j = \sum_{j>i} z_j \frac{\langle \mathbf{b}_j, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle} = \frac{\langle \sum_{j>i} z_j \mathbf{b}_j, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle}.$$

It remains to show that $t_i + \sum_{j>i} \mathbf{U}_{ij} t_j = \frac{\langle \mathbf{c}, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle}$. Since $\tilde{\mathbf{B}}$ is full-rank and orthogonal,

$$\mathbf{c} = \sum_{i=1}^n \frac{\langle \mathbf{c}, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle} \tilde{\mathbf{b}}_i = \tilde{\mathbf{B}} \left[\frac{\langle \mathbf{c}, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle} \right]_i.$$

Therefore,

$$\left[\frac{\langle \mathbf{c}, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle} \right]_i = \tilde{\mathbf{B}}^{-1} \mathbf{c} = \tilde{\mathbf{B}}^{-1} \mathbf{B} \mathbf{t} = \mathbf{U} \mathbf{t} = \left[\sum_{j=1}^n \mathbf{U}_{ij} t_j \right]_i = \left[t_i + \sum_{j>i} \mathbf{U}_{ij} t_j \right]_i.$$

□

Corollary 1 (Equivalent Formulation of Theorem 1). *Let $\mathbf{t}' = (t'_1, \dots, t'_n)$. Then*

$$\mathbf{B}(\mathbf{z} - \mathbf{t}) = \tilde{\mathbf{B}}(\mathbf{z} - \mathbf{t}') \in \tilde{\mathbf{B}} \cdot \left[-\frac{1}{2}, \frac{1}{2} \right]^n.$$

Proof.

$$\begin{aligned}
\tilde{\mathbf{B}}(\mathbf{z} - \mathbf{t}') &= \sum_{i=1}^n (z_i - t'_i) \tilde{\mathbf{b}}_i \\
&= \sum_{i=1}^n \left(z_i - t_i - \sum_{j>i} \mathbf{U}_{ij}(t_j - z_j) \right) \tilde{\mathbf{b}}_i \\
&= \sum_{i=1}^n \left(\sum_{j \geq i} \mathbf{U}_{ij}(z_j - t_j) \right) \tilde{\mathbf{b}}_i \\
&= \sum_{j=1}^n (z_j - t_j) \sum_{i < j} \mathbf{U}_{ij} \tilde{\mathbf{b}}_i \\
&= \sum_{j=1}^n (z_j - t_j) \mathbf{b}_j \\
&= \mathbf{B}(\mathbf{z} - \mathbf{t}).
\end{aligned}$$

□

5 Randomized Version [GPV08]

Algorithm 4 Randomized Nearest-Plane (Discrete Gaussian Sampler)

Input: Basis $\mathbf{B}$, target vector $\mathbf{c}$, parameter σ

Output: Vector $\mathbf{v}_0 \in \mathcal{L}(\mathbf{B})$ distributed as $D_{\mathcal{L}(\mathbf{B}), \sigma, \mathbf{c}}$

```

1:  $\mathbf{c}_n \leftarrow \mathbf{c}$ ,  $\mathbf{v}_n \leftarrow \mathbf{0}$ 
2: for  $i = n, \dots, 1$  do
3:    $c'_i \leftarrow \frac{\langle \mathbf{c}_i, \tilde{\mathbf{b}}_i \rangle}{\langle \tilde{\mathbf{b}}_i, \tilde{\mathbf{b}}_i \rangle}$ ,  $\sigma_i \leftarrow \frac{\sigma}{\|\tilde{\mathbf{b}}_i\|}$ 
4:    $z_i \leftarrow D_{\mathbb{Z}, \sigma_i, c'_i}$ 
5:    $\mathbf{c}_{i-1} \leftarrow \mathbf{c}_i - z_i \mathbf{b}_i$ ,  $\mathbf{v}_{i-1} \leftarrow \mathbf{v}_i + z_i \mathbf{b}_i$ 
6: end for
7: return  $\mathbf{v}_0$ 

```

Algorithm 5 Randomized Nearest-Plane via $\mathbf{U}$

Input: Basis $\mathbf{B}$, target vector $\mathbf{c}$, parameter σ

Output: Vector $\mathbf{v} \in \mathcal{L}(\mathbf{B})$ distributed as $D_{\mathcal{L}(\mathbf{B}), \sigma, \mathbf{c}}$

```

1: Find  $\mathbf{t} = \mathbf{B}^{-1}\mathbf{c}$  such that  $\mathbf{B}\mathbf{t} = \mathbf{c}$ 
2: for  $i = n, \dots, 1$  do
3:    $t'_i \leftarrow t_i + \sum_{j>i} \mathbf{U}_{ij}(t_j - z_j)$ 
4:    $\sigma_i \leftarrow \frac{\sigma}{\|\tilde{\mathbf{b}}_i\|}$ 
5:    $z_i \leftarrow D_{\mathbb{Z}, \sigma_i, t'_i}$ 
6: end for
7: return  $\mathbf{v} = \sum_{i=1}^n z_i \mathbf{b}_i$ 

```

Note that Theorem 1 also holds for the randomized algorithms above.

Theorem 2 (Lemma 4.5 in [GPV08]). *For $\sigma \geq \|\tilde{\mathbf{B}}\| \cdot \omega(\sqrt{\log n}) = \max_i \|\tilde{\mathbf{b}}_i\| \cdot \omega(\sqrt{\log n})$,*

$$\mathbf{v}_0 \stackrel{\Delta}{\sim} D_{\mathcal{L}(\mathbf{B}), \sigma, \mathbf{c}}.$$

Concretely, we require $\sigma \geq \|\tilde{\mathbf{B}}\| \cdot \eta_\epsilon(\mathbb{Z})$, which ensures $\sigma_i \geq \eta_\epsilon(\mathbb{Z})$ for all i .

Proof. Let $\mathbf{v}_0 = \sum_{i=1}^n z_i \mathbf{b}_i$ where $z_i \leftarrow D_{\mathbb{Z}, \sigma_i, c'_i}$. The probability of outputting $\mathbf{v}_0$ is

$$\prod_{i=1}^n D_{\mathbb{Z}, \sigma_i, c'_i}(z_i) = \frac{\prod \rho_{\sigma_i, c'_i}(z_i)}{\prod \rho_{\sigma_i, c'_i}(\mathbb{Z})}.$$

Note that $\rho_{\sigma'_i, c'_i}(x) = \rho_{\sigma'_i}(x - c'_i) = \rho_{\sigma}((x - c'_i) \cdot \|\tilde{\mathbf{b}}_i\|)$. The numerator equals

$$\prod_{i=1}^n \rho_{\sigma'_i, c'_i}(z_i) = \prod_{i=1}^n \rho_{\sigma}((z_i - c'_i) \cdot \|\tilde{\mathbf{b}}_i\|) = \rho_{\sigma} \left(\sum_{i=1}^n (z_i - c'_i) \cdot \tilde{\mathbf{b}}_i \right) = \rho_{\sigma}(\mathbf{v}_0 - \mathbf{c}) = \rho_{\sigma, \mathbf{c}}(\mathbf{v}_0).$$

Now we need the denominator to be close to $\rho_{\sigma, \mathbf{c}}(\mathcal{L}(\mathbf{B}))$. Recall the smoothing-parameter lemma: since $\sigma_i \geq \omega(\sqrt{\log n})$, $\sigma_i \geq \eta_{\epsilon}(\mathbb{Z})$ for negligible $\epsilon(n)$, and

$$\rho_{\sigma'_i, c'_i}(\mathbb{Z}) \in \left[\frac{1 - \epsilon}{1 + \epsilon}, 1 \right] \cdot \rho_{\sigma'_i}(\mathbb{Z}).$$

Therefore,

$$\prod \rho_{\sigma'_i, c'_i}(\mathbb{Z}) \in \left[\left(\frac{1 - \epsilon}{1 + \epsilon} \right)^n, 1 \right] \cdot \prod \rho_{\sigma'_i}(\mathbb{Z}).$$

This range no longer depends on c'_i or $\mathbf{c}$. In particular, the probability of outputting $\mathbf{v}_0$ equals

$$\Pr[\mathbf{v}_0] = \frac{\rho_{\sigma, \mathbf{c}}(\mathbf{v}_0)}{\prod \rho_{\sigma'_i, c'_i}(\mathbb{Z})} \in \rho_{\sigma, \mathbf{c}}(\mathbf{v}_0) \cdot \left[1, \left(\frac{1 + \epsilon}{1 - \epsilon} \right)^n \right] \cdot \frac{1}{\prod \rho_{\sigma'_i}(\mathbb{Z})}.$$

Since summing over all possible $\mathbf{v}_0$ gives 1,

$$\prod \rho_{\sigma'_i}(\mathbb{Z}) \in \left[1, \left(\frac{1 + \epsilon}{1 - \epsilon} \right)^n \right] \cdot \rho_{\sigma, \mathbf{c}}(\mathcal{L}(\mathbf{B})).$$

So the probability of outputting $\mathbf{v}_0$ is within $\left[\left(\frac{1 - \epsilon}{1 + \epsilon} \right)^n, \left(\frac{1 + \epsilon}{1 - \epsilon} \right)^n \right]$ of $D_{\mathcal{L}(\mathbf{B}), \sigma, \mathbf{c}}(\mathbf{v}_0)$. For negligible ϵ , $\left(\frac{1 + \epsilon}{1 - \epsilon} \right)^n$ is also negligible, so the statistical distance is negligible.

Concretely, the statistical distance is smaller than

$$\frac{1}{2} \left[\left(\frac{1 + \epsilon}{1 - \epsilon} \right)^n - 1 \right] = \frac{1}{2} \epsilon'(\epsilon, n),$$

where $\epsilon' \approx 2n\epsilon$. Letting the statistical distance $\frac{1}{2}\epsilon' = 2^{-\lambda}$ be fixed (for a given security parameter λ), the requirement on σ becomes

$$\sigma \geq \|\tilde{\mathbf{B}}\| \cdot \eta_{2^{-\lambda}}(\mathbb{Z}^n),$$

because we require

$$\sigma_i \geq \eta_{\epsilon}(\mathbb{Z}) \approx \eta_{n\epsilon}(\mathbb{Z}^n) \approx \eta_{\frac{1}{2}\epsilon'}(\mathbb{Z}^n) = \eta_{2^{-\lambda}}(\mathbb{Z}^n).$$

This matches the theorem in [Pre17]. □

Remark 1. This also implies that by calling this algorithm multiple times, one can obtain the “closest” point with high probability.

6 Fast Fourier Orthogonalization [DP16]

In consistency with the paper, we use $\mathbf{B} = \mathbf{L}\tilde{\mathbf{B}}$ for some orthogonal $\tilde{\mathbf{B}} = \{\tilde{\mathbf{b}}_1, \dots, \tilde{\mathbf{b}}_n\}$ and unit lower-triangular matrix $\mathbf{L}$, where the $\tilde{\mathbf{b}}_i$ are row vectors,

$$\tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j=1}^{i-1} \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle} \tilde{\mathbf{b}}_j, \quad \mathbf{L} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ \frac{\langle \mathbf{b}_2, \tilde{\mathbf{b}}_1 \rangle}{\langle \tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_1 \rangle} & 1 & 0 & \cdots & 0 \\ \frac{\langle \mathbf{b}_3, \tilde{\mathbf{b}}_1 \rangle}{\langle \tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_1 \rangle} & \frac{\langle \mathbf{b}_3, \tilde{\mathbf{b}}_2 \rangle}{\langle \tilde{\mathbf{b}}_2, \tilde{\mathbf{b}}_2 \rangle} & 1 & \cdots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\langle \mathbf{b}_n, \tilde{\mathbf{b}}_1 \rangle}{\langle \tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_1 \rangle} & \frac{\langle \mathbf{b}_n, \tilde{\mathbf{b}}_2 \rangle}{\langle \tilde{\mathbf{b}}_2, \tilde{\mathbf{b}}_2 \rangle} & \cdots & 0 & 1 \end{bmatrix}.$$

6.1 Definitions

Let $R_d = \mathbb{R}[X]/(X^d - 1)$ for $d = 2^h$; we call this the circular convolution ring. Let $a = \sum_{i \in \mathbb{Z}_d} a_i X^i \in R_d$.

Definition 4 (Adjoint).

$$a^*(X) = a(1/X) \bmod (X^d - 1) = a_0 + \sum_{i=1}^{d-1} a_i X^{d-i}.$$

Let $\Omega = \{\zeta \mid \zeta^d - 1 = 0\}$. Then a^* is the unique element of R_d such that $a^*(\zeta) = \overline{a(\zeta)}$ for all $\zeta \in \Omega$.

Definition 5 (Inner Product).

$$\langle a, b \rangle = \sum_{\zeta^d - 1 = 0} a(\zeta) \cdot \overline{b(\zeta)}, \quad \|a\| = \sqrt{\langle a, a \rangle}.$$

This differs from the canonical inner product $\langle a, b \rangle_2 = \sum_i a_i b_i$. In fact, $\{a(\zeta)\}_{\zeta^d - 1 = 0} = [a_0, \dots, a_{d-1}] \mathbf{V}_d$, where $\mathbf{V}_d = (\zeta_d^{ij})_{i,j}$ is the Vandermonde matrix. Therefore, if $a, b \in R_d$ (hence real-coefficient),

$$\langle a, b \rangle = [a_0, \dots, a_{d-1}] \mathbf{V}_d \cdot \mathbf{V}_d^* \cdot [b_0, \dots, b_{d-1}] = d \cdot \langle a, b \rangle_2.$$

Definition 6 (Coefficient Maps).

1. The coefficient vector $c(a) = (a_0, \dots, a_{d-1})$.
2. The circulant matrix

$$C(a) = \begin{bmatrix} a_0 & a_1 & \dots & a_{d-1} \\ a_{d-1} & a_0 & \dots & a_{d-2} \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \dots & a_0 \end{bmatrix} = \begin{bmatrix} c(a) \\ c(Xa) \\ \vdots \\ c(X^{d-1}a) \end{bmatrix} \in \mathbb{R}^{d \times d}.$$

3. $C(a)C(b) = C(ab)$.
4. $c(a)C(b) = c(ab)$.
5. $C(a)^* = C(a^*)$.

6.2 LDL Decomposition $\text{LDL}_{R_d}(\mathbf{G})$

Let $\mathbf{G} = \mathbf{B}\mathbf{B}^*$. The LDL decomposition is $\mathbf{G} = \mathbf{L}\mathbf{D}\mathbf{L}^*$, where $\mathbf{D} = \tilde{\mathbf{B}}\tilde{\mathbf{B}}^*$ and $\mathbf{B} = \mathbf{L}\tilde{\mathbf{B}}$.

Algorithm 6 $\text{LDL}_{R_d}(\mathbf{G} \in R_d^{n \times n})$

Input: Gram matrix $\mathbf{G} \in R_d^{n \times n}$

Output: Unit lower-triangular $\mathbf{L}$, diagonal $\mathbf{D}$ with $\mathbf{G} = \mathbf{L}\mathbf{D}\mathbf{L}^*$

- 1: $\mathbf{L}, \mathbf{D} \leftarrow \mathbf{0}^{n \times n}$
 - 2: **for** $i = 1$ to n **do**
 - 3: $L_{ii} \leftarrow 1$
 - 4: $D_i \leftarrow G_{ii} - \sum_{j < i} L_{ij} L_{ij}^* D_j$
 - 5: **for** $j = 1$ to $i - 1$ **do**
 - 6: $L_{ij} \leftarrow \frac{1}{D_j} \left(G_{ij} - \sum_{k < j} L_{ik} L_{jk}^* D_k \right)$
 - 7: **end for**
 - 8: **end for**
 - 9: **return** $\mathbf{L}, \mathbf{D}$
-

6.3 Nearest-Plane Algorithm $\text{NP}_{\mathbb{R}}(\mathbf{t}, \mathbf{L})$

We rewrite the nearest-plane algorithm in accordance with the notation above. Given a target vector $\mathbf{t}$ and $\mathbf{L}$ such that $\mathbf{B} = \mathbf{L}\tilde{\mathbf{B}}$:

Algorithm 7 $\text{NP}_{\mathbb{R}}(\mathbf{t}, \mathbf{L})$

Input: Target vector $\mathbf{t}$, unit lower-triangular $\mathbf{L}$

Output: $\mathbf{z} \in \mathbb{Z}^n$

```

1: for  $j = n, \dots, 1$  do
2:    $t'_j \leftarrow t_j + \sum_{i>j} (t_i - z_i) \mathbf{L}_{ij}$ 
3:    $z_j \leftarrow \lfloor t'_j \rfloor$ 
4: end for
5: return  $\mathbf{z} = (z_1, \dots, z_n)$ 

```

It is guaranteed that $(\mathbf{t} - \mathbf{z})\mathbf{B} \in [-\frac{1}{2}, \frac{1}{2}]^n \cdot \tilde{\mathbf{B}}$.

6.4 Linearization

Let $d = 2^h$ and $d' = 2^{h'}$ for some $h' \leq h$, and let $k = d/d'$, $Y = X^k$.

Definition 7 ($V_{d/d'} : R_d^m \rightarrow R_{d'}^{km}$).

1. For $d = d' = 1$, $V_{d/d'}$ is the identity.
2. For $d' = \text{gpd}(d) = 2^{h-1}$: given $a \in R_d$, write $a = \sum_{i \in \mathbb{Z}_k} X^i a_i(Y) = a_0(X^2) + X a_1(X^2)$ with $a_i \in R_{d'}$. Then $V_{d/d'}(a) = (a_0, \dots, a_{k-1})$.
3. Extend $V_{d/d'}$ from $m = 1$ to $m > 1$ component-wise.
4. For a tower of proper divisors $d'' \mid d' \mid d$ and any $a \in R_d$,

$$V_{d/d''}(a) = V_{d'/d''} \circ V_{d/d'}(a) \in R_{d''}^{d/d''}.$$

Definition 8 ($M_{d/d'} : R_d^{n \times m} \rightarrow R_{d'}^{kn \times km}$).

1. For $d = d' = 1$, $M_{d/d'}$ is the identity.
2. For $d' = 2^{h-1}$: given $a \in R_d$, write $a = \sum_{i \in \mathbb{Z}_k} X^i a_i(Y)$ with $a_i \in R_{d'}$, and set

$$M_{d/d'}(a) = \begin{bmatrix} a_0 & a_1 & \dots & a_{k-1} \\ Y a_{k-1} & a_0 & \dots & a_{k-2} \\ \vdots & \vdots & \ddots & \vdots \\ Y a_1 & Y a_2 & \dots & a_0 \end{bmatrix} = \begin{bmatrix} V_{d/d'}(a) \\ V_{d/d'}(Y a) \\ \vdots \\ V_{d/d'}(Y^{d'-1} a) \end{bmatrix} = \begin{bmatrix} a_0 & a_1 \\ X^2 a_1 & a_0 \end{bmatrix}.$$

3. Extend $M_{d/d'}$ from $n = m = 1$ to $n, m > 1$ component-wise.
4. For a tower of proper divisors $d'' \mid d' \mid d$ and any $a \in R_d$,

$$M_{d/d''}(a) = M_{d'/d''} \circ M_{d/d'}(a) \in R_{d''}^{d/d'' \times d/d''}.$$

6.4.1 Example

Let $c(a) = (0, 1, 2, 3, 4, 5, 6, 7)$, so $a_0 = (0, 2, 4, 6)$ and $a_1 = (1, 3, 5, 7)$:

$$C(a) = \begin{bmatrix} 0 & 1 & \dots & 7 \\ 7 & 0 & \dots & 6 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 2 & \dots & 0 \end{bmatrix}.$$

Then $c(V_{8/4}(a)) = ((0, 2, 4, 6), (1, 3, 5, 7))$ and

$$M_{8/4}(a) = \begin{bmatrix} (0, 2, 4, 6) & (1, 3, 5, 7) \\ X^2(1, 3, 5, 7) & (0, 2, 4, 6) \end{bmatrix} = \begin{bmatrix} (0, 2, 4, 6) & (1, 3, 5, 7) \\ (7, 1, 3, 5) & (0, 2, 4, 6) \end{bmatrix},$$

$$C(M_{8/4}(a)) = \begin{bmatrix} 0 & 2 & 4 & 6 & 1 & 3 & 5 & 7 \\ 6 & 0 & 2 & 4 & 7 & 1 & 3 & 5 \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ 7 & 1 & 3 & 5 & 0 & 2 & 4 & 6 \\ 5 & 7 & 1 & 3 & 6 & 0 & 2 & 4 \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \end{bmatrix}.$$

Similarly, $c(V_{8/2}(a)) = (((0, 4), (2, 6)), ((1, 5), (3, 7)))$ and

$$M_{8/2}(a) = \begin{bmatrix} \begin{bmatrix} (0, 4) & (2, 6) \\ (6, 2) & (0, 4) \end{bmatrix} & \begin{bmatrix} (1, 5) & (3, 7) \\ (7, 3) & (1, 5) \end{bmatrix} \\ \begin{bmatrix} (7, 3) & (1, 5) \\ (5, 1) & (7, 3) \end{bmatrix} & \begin{bmatrix} (0, 4) & (2, 6) \\ (6, 2) & (0, 4) \end{bmatrix} \end{bmatrix},$$

$$M_{8/1}(a) = C(M_{8/2}(a)) = \begin{bmatrix} \begin{bmatrix} 0 & 4 \\ 4 & 0 \end{bmatrix} & \begin{bmatrix} 2 & 6 \\ 6 & 2 \end{bmatrix} & \begin{bmatrix} 1 & 5 \\ 5 & 1 \end{bmatrix} & \begin{bmatrix} 3 & 7 \\ 7 & 3 \end{bmatrix} \\ \begin{bmatrix} 6 & 2 \\ 2 & 6 \end{bmatrix} & \begin{bmatrix} 0 & 4 \\ 4 & 0 \end{bmatrix} & \begin{bmatrix} 7 & 3 \\ 3 & 7 \end{bmatrix} & \begin{bmatrix} 1 & 5 \\ 5 & 1 \end{bmatrix} \\ \begin{bmatrix} 7 & 3 \\ 3 & 7 \end{bmatrix} & \begin{bmatrix} 1 & 5 \\ 5 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 4 \\ 4 & 0 \end{bmatrix} & \begin{bmatrix} 2 & 6 \\ 6 & 2 \end{bmatrix} \\ \begin{bmatrix} 5 & 1 \\ 1 & 5 \end{bmatrix} & \begin{bmatrix} 7 & 3 \\ 3 & 7 \end{bmatrix} & \begin{bmatrix} 6 & 2 \\ 2 & 6 \end{bmatrix} & \begin{bmatrix} 0 & 4 \\ 4 & 0 \end{bmatrix} \end{bmatrix}.$$

6.4.2 Properties

1. $M_{d/d'}(\mathbf{A} \cdot \mathbf{B}) = M_{d/d'}(\mathbf{A})M_{d/d'}(\mathbf{B})$.
2. $V_{d/d'}(ab) = V_{d/d'}(a) \cdot M_{d/d'}(b)$.
3. $\langle V(\mathbf{a}), V(\mathbf{b}) \rangle = \langle \mathbf{a}, \mathbf{b} \rangle_2$, the canonical inner product.
4. $V_{d/d'}$ is injective and linear.
5. $M_{d/d'}(\mathbf{B})$ is full-rank iff $\mathbf{B}$ is full-rank.

Proof. One checks directly that $M_{d/d'}(a) \cdot M_{d/d'}(b) = M_{d/d'}(a \cdot b)$. Let $a = \sum_{i \in \mathbb{Z}_k} X^i a_i(Y)$ and $b = \sum_{i \in \mathbb{Z}_k} X^i b_i(Y)$; then

$$a \cdot b = \sum_{i \in \mathbb{Z}_k} X^i \cdot \left(\sum_{\substack{j_1 + j_2 = i \\ j_1, j_2 \in \mathbb{Z}_k}} a_{j_1} b_{j_2} + \sum_{\substack{j_1 + j_2 = k + i \\ j_1, j_2 \in \mathbb{Z}_k}} a_{j_1} b_{j_2} \cdot Y \right).$$

□

6.4.3 Computation in FFT

For any $a \in R_d$, $\mathbf{a} \in V_{d/d'}(R_d)$, $\mathbf{A} \in M_{d/d'}(R_d)$:

- $V_{d/d'}(a)$, $V_{d/d'}^{-1}(\mathbf{a})$, $M_{d/d'}^{-1}(\mathbf{A})$ can all be computed in FFT form in time $\Theta(kd)$.
- $M_{d/d'}(a)$ can be computed in FFT form in time $\Theta(k^2d)$.

This holds because the coefficients of $V_{d/d'}(a)$ and $M_{d/d'}(a)$ are exactly the a_i 's, where $a = \sum_{i \in \mathbb{Z}_k} X^i a_i(Y) = a_0(X^2) + X a_1(X^2)$. One can switch from the FFT of a to the FFTs of all a_i 's in time $\Theta(kd)$.

6.5 LDL Decomposition in R_d^m

Let $1 = d_0 \mid d_1 \mid \dots \mid d_h = d$ (i.e. $d_i = 2^i$) be a tower of proper divisors of d .

Theorem 3. Let $\mathbf{b} \in R_d^m$ be such that $M_{d/1}(\mathbf{b})$ is full-rank. Then there is a GSO of $M_{d/1}(\mathbf{b})$

$$M_{d/1}(\mathbf{b}) = \left(\prod_{i=0}^{h-1} M_{d_i/1}(\mathbf{L}_i) \right) \cdot \tilde{\mathbf{B}}_0,$$

where

- $\tilde{\mathbf{B}}_0 \in \mathbb{R}^{d \times dm}$ is orthogonal;
- $\mathbf{L}_i \in R_{d_i}^{(d/d_i) \times (d/d_i)}$ is a block-diagonal matrix whose diagonal blocks are unit lower-triangular matrices of $R_{d_i}^{(d_{i+1}/d_i) \times (d_{i+1}/d_i)}$, with d/d_{i+1} such blocks;
- the multiplication order is $M_{d_{h-1}/1}(\mathbf{L}_{h-1}) \cdots M_{d_1/1}(\mathbf{L}_1) \cdot M_{1/1}(\mathbf{L}_0) \cdot \tilde{\mathbf{B}}_0$.

6.5.1 Example

For $a \in R_8$, $M_{8/1}(a)$ decomposes as

$$M_{8/1}(a) = M_{4/1}(\mathbf{L}_2) \cdot M_{2/1}(\mathbf{L}_1) \cdot M_{1/1}(\mathbf{L}_0) \cdot \tilde{\mathbf{B}}_0,$$

where

- $\mathbf{L}_2 \in R_{d_2}^{d/d_2 \times d/d_2} = R_4^{2 \times 2}$ has $d/d_3 = 1$ diagonal block in $R_{d_2}^{d_2/d_1 \times d_2/d_1} = R_4^{2 \times 2}$;
- $\mathbf{L}_1 \in R_{d_1}^{d/d_1 \times d/d_1} = R_2^{4 \times 4}$ has $d/d_2 = 2$ diagonal blocks in $R_{d_1}^{d_2/d_1 \times d_2/d_1} = R_2^{2 \times 2}$;
- $\mathbf{L}_0 \in R_{d_0}^{d/d_0 \times d/d_0} = \mathbb{R}^{8 \times 8}$ has $d/d_1 = 4$ diagonal blocks in $R_{d_0}^{d_1/d_0 \times d_1/d_0} = \mathbb{R}^{2 \times 2}$;
- $\tilde{\mathbf{B}}_0 \in \mathbb{R}^{d \times d} = \mathbb{R}^{8 \times 8}$ is orthogonal.

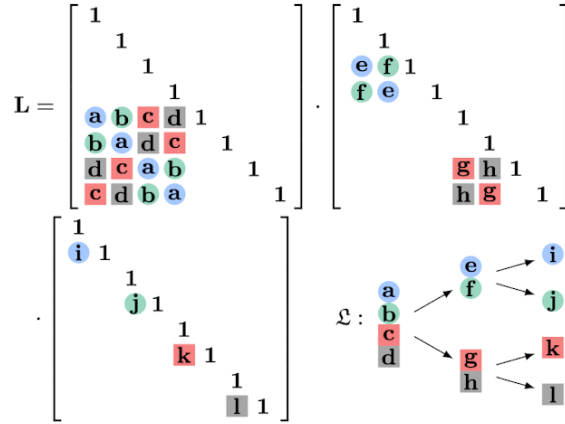


Figure 2: The factorization tree $\mathbf{L}_2, \mathbf{L}_1, \mathbf{L}_0, \tilde{\mathbf{B}}_0$ for $d = 8$, from [DP16].

Proof. We proceed by induction on d . When d is prime this is the regular GSO. Suppose d is composite and the claim holds for all $R_{d'}$ with $d' < d$.

By the properties above, $\mathbf{B}_{h-1} := M_{d/d_{h-1}}(\mathbf{b})$ is full-rank, so it decomposes as $\mathbf{B}_{h-1} = \mathbf{L}_{h-1} \tilde{\mathbf{B}}$, where $\mathbf{L}_{h-1} \in R_{d_{h-1}}^{d/d_{h-1} \times d/d_{h-1}}$ is unit lower-triangular and $\tilde{\mathbf{B}} \in R_{d_{h-1}}^{d/d_{h-1} \times md/d_{h-1}}$ is orthogonal. Write $\tilde{\mathbf{B}} = [\mathbf{b}_1, \dots, \mathbf{b}_{d/d_{h-1}}]$ as row vectors in $R_{d_{h-1}}^{md/d_{h-1}}$. By the induction hypothesis, each decomposes as

$$M_{d_{h-1}/1}(\mathbf{b}_j) = \left(\prod_{i=0}^{h-2} M_{d_i/1}(\mathbf{L}_{i,j}) \right) \cdot \tilde{\mathbf{B}}_j \quad \forall j \in [d/d_{h-1}],$$

where $\tilde{\mathbf{B}}_j \in \mathbb{R}^{d_{h-1} \times md}$ is orthogonal and $\mathbf{L}_{i,j} \in R_{d_i}^{(d_{h-1}/d_i) \times (d_{h-1}/d_i)}$ is block-diagonal with d_{h-1}/d_{i+1} diagonal blocks in $R_{d_i}^{(d_{i+1}/d_i) \times (d_{i+1}/d_i)}$.

Therefore,

$$\begin{aligned} M_{d/1}(\mathbf{b}) &= M_{d_{h-1}/1}(M_{d/d_{h-1}}(\mathbf{b})) = M_{d_{h-1}/1}(\mathbf{B}_{h-1}) \\ &= M_{d_{h-1}/1}(\mathbf{L}_{h-1}) \cdot M_{d_{h-1}/1}(\tilde{\mathbf{B}}) \\ &= M_{d_{h-1}/1}(\mathbf{L}_{h-1}) \cdot [M_{d_{h-1}/1}(\mathbf{b}_1), \dots, M_{d_{h-1}/1}(\mathbf{b}_{d/d_{h-1}})] \\ &= M_{d_{h-1}/1}(\mathbf{L}_{h-1}) \cdot \left[\left(\prod_{i=0}^{h-2} M_{d_i/1}(\mathbf{L}_{i,j}) \right) \cdot \tilde{\mathbf{B}}_j \right]_{j \in [d/d_{h-1}]}. \end{aligned}$$

Let $\mathbb{L}_{i,j} := M_{d_i/1}(\mathbf{L}_{i,j}) \in \mathbb{R}^{d_{h-1} \times d_{h-1}}$ and $\tilde{\mathbf{B}}_0 := [\tilde{\mathbf{B}}_1, \dots, \tilde{\mathbf{B}}_{d/d_{h-1}}]$. Then

$$M_{d/1}(\mathbf{b}) = M_{d_{h-1}/1}(\mathbf{L}_{h-1}) \cdot \left(\prod_{i=0}^{h-2} \text{Diag}(\{\mathbb{L}_{i,j}\}_j) \right) \cdot \tilde{\mathbf{B}}_0,$$

where

$$\text{Diag}(\{\mathbb{L}_{i,j}\}_j) = \begin{bmatrix} \mathbb{L}_{i,1} & 0 & \cdots & 0 \\ 0 & \mathbb{L}_{i,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbb{L}_{i,d/d_{h-1}} \end{bmatrix}.$$

Setting

$$\mathbf{L}_i := \text{Diag}(\{\mathbf{L}_{i,j}\}_j) = \begin{bmatrix} \mathbf{L}_{i,1} & 0 & \cdots & 0 \\ 0 & \mathbf{L}_{i,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{L}_{i,d/d_{h-1}} \end{bmatrix},$$

we have $\text{Diag}(\{\mathbb{L}_{i,j}\}_j) = M_{d_i/1}(\mathbf{L}_i)$, so

$$M_{d/1}(\mathbf{b}) = M_{d_{h-1}/1}(\mathbf{L}_{h-1}) \cdot \left(\prod_{i=0}^{h-2} M_{d_i/1}(\mathbf{L}_i) \right) \cdot \tilde{\mathbf{B}}_0 = \left(\prod_{i=0}^{h-1} M_{d_i/1}(\mathbf{L}_i) \right) \cdot \tilde{\mathbf{B}}_0.$$

Finally we check the requirements. $\mathbf{L}_i$ has d/d_{h-1} blocks in $R_{d_i}^{(d_{h-1}/d_i) \times (d_{h-1}/d_i)}$, each with d_{h-1}/d_{i+1} blocks in $R_{d_i}^{(d_{i+1}/d_i) \times (d_{i+1}/d_i)}$, hence d/d_{i+1} such blocks overall, as required.

For orthogonality of $\tilde{\mathbf{B}}_0 = [\tilde{\mathbf{B}}_1, \dots, \tilde{\mathbf{B}}_{d/d_{h-1}}]$: two row vectors $\mathbf{u}, \mathbf{v}$ within the same $\tilde{\mathbf{B}}_j$ are orthogonal by the induction hypothesis. If $\mathbf{u}$ belongs to $\tilde{\mathbf{B}}_j$ and $\mathbf{v}$ to $\tilde{\mathbf{B}}_\ell$ ($j \neq \ell$), write $\mathbf{u} = \mathbf{a}_j \cdot M_{d_{h-1}/1}(\mathbf{b}_j)$ and $\mathbf{v} = \mathbf{a}_\ell \cdot M_{d_{h-1}/1}(\mathbf{b}_\ell)$ for some $\mathbf{a}_j, \mathbf{a}_\ell \in \mathbb{R}^{d_{h-1}}$, and let $a_j = V_{d_{h-1}/1}^{-1}(\mathbf{a}_j)$, $a_\ell = V_{d_{h-1}/1}^{-1}(\mathbf{a}_\ell)$. By the properties above,

$$\begin{aligned} \langle \mathbf{u}, \mathbf{v} \rangle_2 &= \langle \mathbf{a}_j \cdot M_{d_{h-1}/1}(\mathbf{b}_j), \mathbf{a}_\ell \cdot M_{d_{h-1}/1}(\mathbf{b}_\ell) \rangle_2 \\ &= \langle V_{d_{h-1}/1}(a_j) \cdot M_{d_{h-1}/1}(\mathbf{b}_j), V_{d_{h-1}/1}(a_\ell) \cdot M_{d_{h-1}/1}(\mathbf{b}_\ell) \rangle_2 \\ &= \langle V_{d_{h-1}/1}(a_j \cdot \mathbf{b}_j), V_{d_{h-1}/1}(a_\ell \cdot \mathbf{b}_\ell) \rangle_2 \\ &= \langle a_j \cdot \mathbf{b}_j, a_\ell \cdot \mathbf{b}_\ell \rangle_2 = 0. \end{aligned}$$

□

6.6 LDL Decomposition in $R_d^{n \times m}$

Theorem 4. Let $1 = d_0 \mid d_1 \mid \dots \mid d_h = d$ be a tower of proper divisors of d , and let $\mathbf{B} \in R_d^{n \times m}$ be full-rank. Then there is a GSO of $M_{d/1}(\mathbf{B})$

$$M_{d/1}(\mathbf{B}) = \left(\prod_{i=0}^h M_{d_i/1}(\mathbf{L}_i) \right) \cdot \tilde{\mathbf{B}}_0 = \mathbf{L} \cdot \tilde{\mathbf{B}}_0,$$

where $\tilde{\mathbf{B}}_0 \in \mathbb{R}^{nd \times md}$ is orthogonal and $\mathbf{L}_i \in R_{d_i}^{(n(d/d_i)) \times (n(d/d_i))}$ is block-diagonal, each block a unit lower-triangular matrix of $R_{d_i}^{(d_{i+1}/d_i) \times (d_{i+1}/d_i)}$, with $n(d/d_{i+1})$ diagonal blocks. Moreover, the LDL decomposition of $M_{d/1}(\mathbf{B}\mathbf{B}^*)$ is $\mathbf{L} \cdot (\tilde{\mathbf{B}}_0 \tilde{\mathbf{B}}_0^t) \cdot \mathbf{L}^t$.

Proof. Let $\mathbf{B} = \mathbf{L}_h \mathbf{B}'$, where $\mathbf{L}_h$ is unit lower-triangular and $\mathbf{B}' = [\mathbf{b}'_1, \dots, \mathbf{b}'_n]$ is orthogonal. Applying Theorem 3 to each $\mathbf{b}'_j$ gives

$$M_{d/1}(\mathbf{b}'_j) = \left(\prod_{i=0}^{h-1} M_{d_i/1}(\mathbf{L}_{i,j}) \right) \cdot \tilde{\mathbf{B}}_j \quad \forall j \in [n].$$

Setting $\tilde{\mathbf{B}}_0 = [\tilde{\mathbf{B}}_1, \dots, \tilde{\mathbf{B}}_n]$ and $\mathbf{L}_i = \text{Diag}(\{\mathbf{L}_{i,j}\}_j)$,

$$\begin{aligned} M_{d/1}(\mathbf{B}) &= M_{d/1}(\mathbf{L}_h) \cdot M_{d/1}([\mathbf{b}'_1, \dots, \mathbf{b}'_n]) \\ &= M_{d/1}(\mathbf{L}_h) \cdot [M_{d/1}(\mathbf{b}'_1), \dots, M_{d/1}(\mathbf{b}'_n)] \\ &= M_{d/1}(\mathbf{L}_h) \cdot \left[\left(\prod_{i=0}^{h-1} M_{d_i/1}(\mathbf{L}_{i,j}) \right) \cdot \tilde{\mathbf{B}}_j \right]_{j \in [n]} \\ &= M_{d/1}(\mathbf{L}_h) \cdot \left(\prod_{i=0}^{h-1} M_{d_i/1}(\mathbf{L}_i) \right) \cdot \tilde{\mathbf{B}}_0 \\ &= \left(\prod_{i=0}^h M_{d_i/1}(\mathbf{L}_i) \right) \cdot \tilde{\mathbf{B}}_0. \end{aligned}$$

□

This shows that for any full-rank $\mathbf{B} \in R_d^{n \times m}$, the GSO of $M_{d/1}(\mathbf{B})$ — equivalently the LDL decomposition of $M_{d/1}(\mathbf{B}\mathbf{B}^*)$ — factors, with each $\mathbf{L}_i$ a unit lower-triangular, sparse, block-diagonal matrix.

6.7 Decomposition Algorithm

Since $V_{d/d'}$ and $M_{d/d'}$ compute efficiently in FFT form, we obtain the fast Fourier GSO algorithm.

Algorithm 8 ffGSO($\mathbf{B} \in R_d^{n \times m}$)

Input: $\mathbf{B} \in R_d^{n \times m}$

Output: Factorization tree $(\mathbf{L}, (\mathcal{L}_i)_i)$

- 1: $(\mathbf{L}, \mathbf{B}') \leftarrow \text{GSO}_{R_d}(\mathbf{B})$
 - 2: **if** $d = 1$ **then**
 - 3: **return** $(\mathbf{L}, \mathbf{B}')$
 - 4: **end if**
 - 5: Let $\mathbf{B}' = [\mathbf{b}'_1, \dots, \mathbf{b}'_n]$
 - 6: $d' \leftarrow \text{gpd}(d)$
 - 7: **for** $i = 1$ to n **do**
 - 8: $\mathcal{L}_i \leftarrow \text{ffGSO}_{R_{d'}}(M_{d/d'}(\mathbf{b}'_i))$
 - 9: **end for**
 - 10: **return** $\mathbf{L}, (\mathcal{L}_i)_{1 \leq i \leq n}$
-

In practice we favor LDL over GSO, since it lets us ignore the row-vector length m , gaining a factor of $\Theta(m)$.

Algorithm 9 fflDL($\mathbf{G} \in R_d^{n \times n}$)

Input: $\mathbf{G} \in R_d^{n \times n}$

Output: Factorization tree $(\mathbf{L}, (\mathcal{L}_i)_i)$

- 1: Transform $\mathbf{G}$ into FFT domain if it is not already
 - 2: $(\mathbf{L}, \mathbf{D}) \leftarrow \text{LDL}_{R_d}(\mathbf{G})$
 - 3: **if** $d = 1$ **then**
 - 4: **return** $(\mathbf{L}, \mathbf{D})$
 - 5: **end if**
 - 6: $d' \leftarrow \text{gpd}(d)$
 - 7: **for** $i = 1$ to n **do**
 - 8: $\mathcal{L}_i \leftarrow \text{ffLDL}_{R_{d'}}(M_{d/d'}(\mathbf{D}_{ii}))$
 - 9: **end for**
 - 10: **return** $\mathbf{L}, (\mathcal{L}_i)_{1 \leq i \leq n}$
-

Note that $\mathbf{D}_{ii} = \mathbf{b}'_i \mathbf{b}_i^{t*}$, as in ffGSO.

6.7.1 Complexity

Let $C(n, d)$ be the complexity of $\text{ffLDL}(\mathbf{G} \in R_d^{n \times n})$. Then

$$C(n, d) = \Theta(n^2 d \log d) + \Theta(n^3 d) + \Theta(d(d/d_{h-1})^2) + n C(d/d_{h-1}, d_{h-1}),$$

where the first term is the FFT of the n^2 coefficients of $\mathbf{G}$ (absent in recursive calls, already in FFT form); the second is $\text{LDL}_{R_d}(\mathbf{G})$ in FFT form (n^3 multiplications in R_d , each costing $\Theta(d)$); the third is computing $M_{d/d_{h-1}}(\mathbf{D}_{ii})$; and the last is the n recursive calls.

Let $k_h = d_h/d_{h-1}$. Expanding the recursion,

$$\begin{aligned} C(k_h, d_{h-1}) &= \sum_{i=1}^h \frac{d}{d_i} \Theta(k_i^3 d_{i-1}) + \sum_{i=1}^h \frac{d}{d_i} \Theta(d_{i-1} k_i^2) + \frac{d}{d_1} C(k_1, 1) \\ &= \sum_{i=1}^h \frac{d}{d_i} \Theta(k_i^3 d_{i-1}) = \sum_{i=1}^h \frac{d}{d_i} \Theta\left(\left(\frac{d_i}{d_{i-1}}\right)^3 d_{i-1}\right) = \Theta(d) \sum_{i=1}^h k_i^2. \end{aligned}$$

Hence

$$C(n, d) = \Theta(n^2 d \log d) + \Theta(n^3 d) + \Theta(nd) \sum_{i=1}^h k_i^2.$$

6.8 Fast Fourier Nearest-Plane $\text{ffNP}_{R_d}(\mathbf{t}, \mathfrak{L})$

Let $Z_d = \mathbb{Z}[X]/(X^d - 1)$. The ffNP algorithm takes a target $\mathbf{t} \in R_d^n$ and a precomputed decomposition tree $\mathfrak{L} \leftarrow \text{ffLDL}(\mathbf{B}\mathbf{B}^*)$.

Algorithm 10 $\text{ffNP}_{R_d}(\mathbf{t}, \mathfrak{L})$

Input: $\mathbf{t} \in R_d^n$, factorization tree $\mathfrak{L}$

Output: $\mathbf{z} \in Z_d^n$

```

1: if  $d = 1$  then
2:    $(\mathbf{L}, \mathbf{D}) \leftarrow \mathfrak{L}$ 
3:   return  $\text{NP}_{\mathbb{R}}(\mathbf{t}, \mathbf{L})$ 
4: end if
5:  $(\mathbf{L}, (\mathfrak{L}_i)_{1 \leq i \leq n}) \leftarrow \mathfrak{L}$ 
6:  $d' \leftarrow \text{gpd}(d)$ 
7: for  $j = n$  to 1 do
8:    $\overline{\mathbf{t}}_j \leftarrow \mathbf{t}_j + \sum_{i>j} (\mathbf{t}_i - \mathbf{z}_i) \mathbf{L}_{ij}$ 
9:    $\mathbf{z}_j \leftarrow V_{d/d'}^{-1}(\text{ffNP}_{R_{d'}}(V_{d/d'}(\overline{\mathbf{t}}_j), \mathfrak{L}_j))$ 
10: end for
11: return  $\mathbf{z} = (\mathbf{z}_1, \dots, \mathbf{z}_n)$ 

```

Lemma 1. For $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_n] \in R_d^{n \times m}$ and its GSO $\tilde{\mathbf{B}} = [\tilde{\mathbf{b}}_1, \dots, \tilde{\mathbf{b}}_n]$ in R_d ,

$$(\mathbf{z} - \mathbf{t}) \cdot \mathbf{B} = (\mathbf{z} - \overline{\mathbf{t}}) \cdot \tilde{\mathbf{B}}.$$

Proof. Recall $\tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j < i} \mathbf{L}_{ij} \tilde{\mathbf{b}}_j$. Then

$$\begin{aligned}
(\mathbf{z} - \bar{\mathbf{t}}) \cdot \tilde{\mathbf{B}} &= \sum_{j=1}^n (\mathbf{z}_j - \bar{\mathbf{t}}_j) \cdot \tilde{\mathbf{b}}_j \\
&= \sum_{j=1}^n \left[\mathbf{z}_j - \left(\mathbf{t}_j + \sum_{i>j} (\mathbf{t}_i - \mathbf{z}_i) \mathbf{L}_{ij} \right) \right] \cdot \tilde{\mathbf{b}}_j \\
&= \sum_{j=1}^n \left[(\mathbf{z}_j - \mathbf{t}_j) + \sum_{i>j} (\mathbf{z}_i - \mathbf{t}_i) \mathbf{L}_{ij} \right] \cdot \tilde{\mathbf{b}}_j \\
&= \sum_{j=1}^n \left[\sum_{i \geq j} (\mathbf{z}_i - \mathbf{t}_i) \mathbf{L}_{ij} \right] \cdot \tilde{\mathbf{b}}_j \\
&= \sum_{i=1}^n (\mathbf{z}_i - \mathbf{t}_i) \cdot \sum_{j \leq i} \mathbf{L}_{ij} \cdot \tilde{\mathbf{b}}_j \\
&= \sum_{i=1}^n (\mathbf{z}_i - \mathbf{t}_i) \cdot \mathbf{b}_i \\
&= (\mathbf{z} - \mathbf{t}) \cdot \mathbf{B}.
\end{aligned}$$

□

Remark 2. If we instead apply $\text{NP}_{\mathbb{R}}$ directly to $c(\mathbf{t}) \in \mathbb{R}^{nd}$, then by the results of the previous section $(c(\mathbf{z}) - c(\mathbf{t})) \cdot C(\mathbf{B}) = (c(\mathbf{z}) - c(\mathbf{t}')) \cdot C(\tilde{\mathbf{B}})$ for some $\mathbf{t}'$. Lemma 1 shows $(c(\mathbf{z}) - c(\mathbf{t})) \cdot C(\mathbf{B}) = (c(\mathbf{z}) - c(\tilde{\mathbf{t}})) \cdot C(\tilde{\mathbf{B}})$, so $c(\mathbf{t}') = c(\tilde{\mathbf{t}})$: running $\text{NP}_{\mathbb{R}}$ on $c(\mathbf{t})$ and $C(\mathbf{B}), C(\tilde{\mathbf{B}})$ computes exactly what ffNP_{R_d} computes — just faster.

The complexity of $\text{NP}_{\mathbb{R}}$ on $c(\mathbf{t}) \in \mathbb{R}^{nd}$ is $\Theta((nd)^2)$, while ffNP_{R_d} on $\mathbf{t} \in R_d^n$ costs $\Theta(n^2d) + \Theta(nd) \sum_{i=1}^h k_i^2$. For Falcon-512 ($n = 2, d = 512$): 1024^2 versus $2048 + 1024 \cdot \sum_{i=1}^9 2 = 2048 + 1024 \cdot 18$.

Theorem 5. Let $\tilde{\mathbf{B}}_0$ be the orthogonalization of $M_{d/1}(\mathbf{B})$ over $\mathbb{R}$. Then

$$V_{d/1}((\mathbf{z} - \mathbf{t})\mathbf{B}) \in \mathcal{P}(\tilde{\mathbf{B}}_0) = \left[-\frac{1}{2}, \frac{1}{2} \right]^n \cdot \tilde{\mathbf{B}}_0.$$

Proof. For $d = 1$ this is exactly the guarantee of $\text{NP}_{\mathbb{R}}(\mathbf{t}, \mathbf{L})$. Assume the claim for all $d' < d$, and let $d' = \text{gpd}(d)$.

By construction $\mathfrak{L}_j = \text{ffGSO}_{R_{d'}}(M_{d/d'}(\mathbf{b}'_j))$, where $\mathbf{B} = \mathbf{L}[\mathbf{b}'_1, \dots, \mathbf{b}'_n]$, and $V_{d/d'}(\mathbf{z}_j) = \text{ffNP}_{R_{d'}}(V_{d/d'}(\bar{\mathbf{t}}_j), \mathfrak{L}_j)$. Writing $\mathbf{B}'_j = M_{d/d'}(\mathbf{b}'_j)$ and letting $\tilde{\mathbf{B}}_j$ be the orthogonalization of $M_{d'/1}(\mathbf{B}'_j) = M_{d/1}(\mathbf{b}'_j)$, the induction hypothesis gives

$$V_{d'/1}((V_{d/d'}(\mathbf{z}_j) - V_{d/d'}(\bar{\mathbf{t}}_j)) \cdot \mathbf{B}'_j) \in \mathcal{P}(\tilde{\mathbf{B}}_j),$$

equivalently

$$V_{d/1}((\mathbf{z}_j - \bar{\mathbf{t}}_j) \cdot \mathbf{b}'_j) \in \mathcal{P}(\tilde{\mathbf{B}}_j).$$

By Lemma 1, $(\mathbf{z} - \mathbf{t}) \cdot \mathbf{B} = \sum_{j=1}^n (\mathbf{z}_j - \bar{\mathbf{t}}_j) \cdot \mathbf{b}'_j$, so

$$V_{d/1}((\mathbf{z} - \mathbf{t}) \cdot \mathbf{B}) \in \mathcal{P}([\tilde{\mathbf{B}}_1, \dots, \tilde{\mathbf{B}}_n]).$$

By Theorem 4, $[\tilde{\mathbf{B}}_1, \dots, \tilde{\mathbf{B}}_n] = \tilde{\mathbf{B}}_0$ is the orthogonalization of $M_{d/1}(\mathbf{B})$, completing the proof. □

6.8.1 Complexity

Let $C(n, d)$ be the complexity of $\text{ffNP}_{R_d}(\mathbf{t}, \mathfrak{L})$ for $\mathbf{t} \in R_d^n$. Then, with $1 = d_0 \mid d_1 \mid \dots \mid d_h = d$ a tower of divisors and $k_h = d_h/d_{h-1}$,

$$C(n, d) = \Theta(nd \log d) + \Theta(n^2d) + \Theta(ndk_h^2) + nC(k_h, d_{h-1}),$$

where the first term is the FFT of the n coefficients of $\mathbf{t}$; the second is the n updates $\overline{\mathbf{t}}_j \leftarrow \mathbf{t}_j + \sum_{i>j} (\mathbf{t}_i - \mathbf{z}_i) \mathbf{L}_{ij}$ in FFT form; the third is computing $V_{d/\text{gcd}(d)}$ and its inverse; and the last is the recursive calls. Expanding the recursion and dropping the (dominated) first term,

$$C(k_h, d_{h-1}) = \sum_{1 \leq i \leq h} \frac{d}{d_i} \Theta(d_{i-1} k_i^3) + \frac{d}{d_1} C(k_1, d_0) = \Theta(d) \sum_{i=1}^h k_i^2,$$

so

$$C(n, d) = \Theta(nd \log d) + \Theta(n^2 d) + \Theta(nd) \sum_{i=1}^h k_i^2.$$

6.9 Application in the Cyclotomic Ring F_d

All of the above transfers from $R_d = \mathbb{R}[X]/(X^d - 1)$ to the cyclotomic ring F_d :

- $\xi_d = e^{2\pi i/d}$, $\Omega_d = \{\xi_d^k \mid k \in \mathbb{Z}_d, \gcd(k, d) = 1\}$.
- $\phi_d(X) = \prod_{\xi \in \Omega_d} (X - \xi) = \prod_{k \in \mathbb{Z}_d^\times} (X - \xi_d^k)$.
- $\psi_d(X) = \prod_{\xi^d=1, \xi \notin \Omega_d} (X - \xi) = \prod_{k \in \mathbb{Z}_d \setminus \mathbb{Z}_d^\times} (X - \xi_d^k)$.
- $\deg(\phi_d) = \varphi(d)$, the Euler totient function of d .
- $\phi_d(X) \psi_d(X) = X^d - 1$.
- $F_d = \mathbb{R}[X]/(\phi_d(X))$.
- When $d = 2^n$, $\phi_d(X) = X^{d/2} + 1$.

Definition 9. Let e_d be the unique element of R_d such that

$$e_d = \begin{cases} 1 \bmod \phi_d \\ 0 \bmod \psi_d. \end{cases}$$

Define the embedding $\iota_d : F_d \rightarrow R_d$ by $\iota_d(f) = f \cdot e_d$, equivalently

$$\iota_d(f)(\xi) = \begin{cases} f(\xi) & \text{if } \phi_d(\xi) = 0, \\ 0 & \text{if } \psi_d(\xi) = 0. \end{cases}$$

With ι_d , define the linear maps

$$V'_{d/d'} := \iota_{d'}^{-1} \circ V_{d/d'} \circ \iota_d, \quad M'_{d/d'} := \iota_{d'}^{-1} \circ M_{d/d'} \circ \iota_d.$$

References

- [Bab86] László Babai. “On Lovász’lattice reduction and the nearest lattice point problem”. In: *Combinatorica* 6.1 (1986), pp. 1–13.
- [Kle00] Philip N. Klein. “Finding the closest lattice vector when it’s unusually close”. In: *11th Annual ACM-SIAM Symposium on Discrete Algorithms*. Ed. by David B. Shmoys. San Francisco, CA, USA: ACM-SIAM, Jan. 2000, pp. 937–941.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. “Trapdoors for hard lattices and new cryptographic constructions”. In: *Proceedings of the fortieth annual ACM symposium on Theory of computing*. 2008, pp. 197–206.
- [DP16] Léo Ducas and Thomas Prest. “Fast fourier orthogonalization”. In: *Proceedings of the 2016 ACM International Symposium on Symbolic and Algebraic Computation*. 2016, pp. 191–198.
- [Pre17] Thomas Prest. “Sharper Bounds in Lattice-Based Cryptography Using the Rényi Divergence”. In: *Advances in Cryptology – ASIACRYPT 2017, Part I*. Ed. by Tsuyoshi Takagi and Thomas Peyrin. Vol. 10624. Lecture Notes in Computer Science. Hong Kong, China: Springer, Cham, Switzerland, Dec. 2017, pp. 347–374. DOI: [10.1007/978-3-319-70694-8_13](https://doi.org/10.1007/978-3-319-70694-8_13).
- [Gue+22] Morgane Guerreau et al. “The Hidden Parallelepiped Is Back Again: Power Analysis Attacks on Falcon”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2022.3 (2022), pp. 141–164. DOI: [10.46586/tches.v2022.i3.141-164](https://doi.org/10.46586/tches.v2022.i3.141-164).